

Министерство науки и высшего образования Российской Федерации
ФГАОУ ВО «Уральский федеральный университет имени первого
Президента России Б.Н. Ельцина»
Институт радиоэлектроники и информационных технологий-РТФ

ОТЧЕТ

По лабораторной работе №2

«Работа с SQLAlchemy и alembic»

По дисциплине «Разработка приложений»

Выполнил: Лапушкин И. А.

Группа: РИМ-150950

Проверил преподаватель:

Кузьмин Д. И.

Екатеринбург

2025

Цель работы: Освоить принципы работы с библиотеками SQLAlchemy и Alembic для создания и управления реляционными базами данных на Python, изучить механизмы миграции базы данных.

Задачи:

1. Настроить рабочее окружение.
2. Разработать ORM-модели данных с использованием SQLAlchemy.
3. Настроить и использовать систему миграций Alembic.
4. Реализовать работу с данными через ORM
5. Проверить корректность работы системы

Ход работы:

1. Подготовка среды разработки

Подготовим локальное окружение и установим зависимости (Рисунок 1).

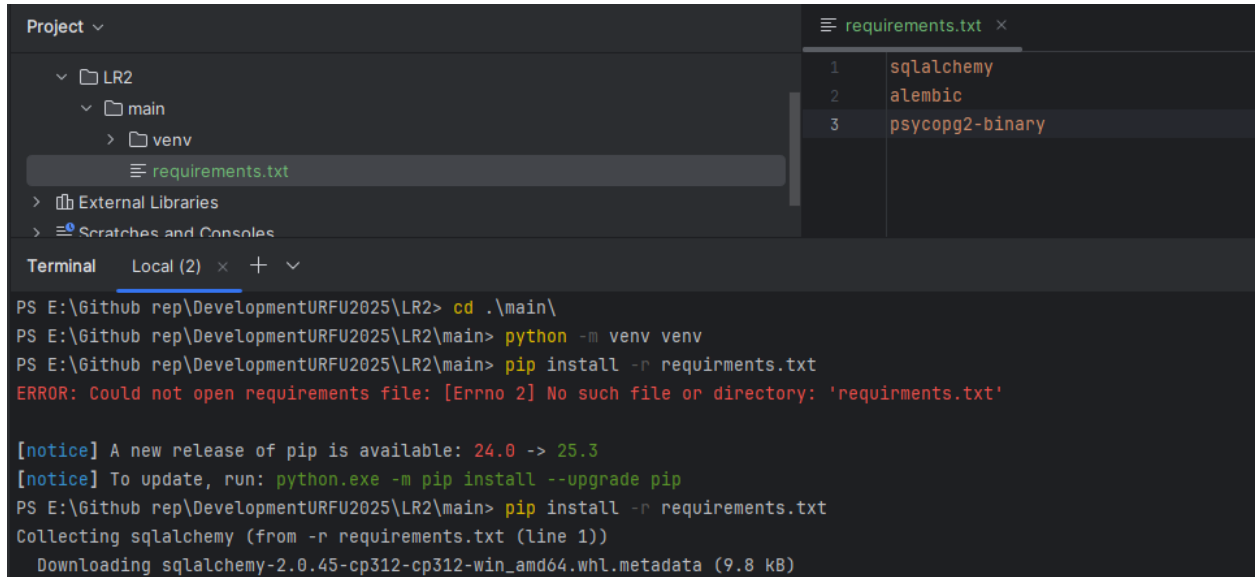


Рисунок 1 – Подготовка

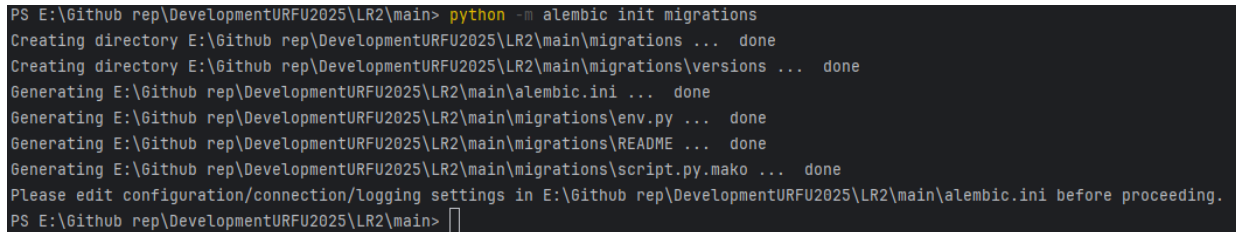
Далее запускаем PostgreSQL в Docker и убеждаемся в работе (Рисунок 2).

```
PS E:\Github rep\DevelopmentURFU2025\LR2\main> docker run --name lr2-base -e POSTGRES_DB=lr2-base -e POSTGRES_USER=postgres -e POSTGRES_PASSWORD=1 -p 5433:5432 -d postgres:15
85471b6f60fc2c8160e562094d9303a6fca173b003ecf268185823c07885078e
PS E:\Github rep\DevelopmentURFU2025\LR2\main>
```

Рисунок 2 – PostgreSQL в Docker

2. Создание и настройка файлов

Создали файл моделей `models.py`. Далее инициализировали Alembic с помощью команды «`python -m alembic init migrations`», появились папка «`migrations`» и файл «`alembic.ini`» (Рисунок 3)



```
PS E:\Github rep\DevelopmentURFU2025\LR2\main> python -m alembic init migrations
Creating directory E:\Github rep\DevelopmentURFU2025\LR2\main\migrations ... done
Creating directory E:\Github rep\DevelopmentURFU2025\LR2\main\migrations\versions ... done
Generating E:\Github rep\DevelopmentURFU2025\LR2\main\alembic.ini ... done
Generating E:\Github rep\DevelopmentURFU2025\LR2\main\migrations\env.py ... done
Generating E:\Github rep\DevelopmentURFU2025\LR2\main\migrations\README ... done
Generating E:\Github rep\DevelopmentURFU2025\LR2\main\migrations\script.py.mako ... done
Please edit configuration/connection/logging settings in E:\Github rep\DevelopmentURFU2025\LR2\main\alembic.ini before proceeding.
PS E:\Github rep\DevelopmentURFU2025\LR2\main> █
```

Рисунок 3 – Запуск команды «`python -m alembic init migrations`»

В файле «`alembic.ini`» нашли строку:

- «`sqlalchemy.url = driver://user:pass@localhost/dbname`»

и заменили ее на

- «`sqlalchemy.url =postgresql://postgres:1@localhost:5433/lr2-base` »

В файле «`migrations/env.py`» нашли строку:

- «`target_metadata = None`»

и заменили ее на:

- «`from models import Base`»
- «`target_metadata = Base.metadata`»

Далее создали и применили миграцию с помощью команд (Рисунок 4):

- «python -m alembic revision --autogenerate -m "init tables"»
- «python -m alembic upgrade head»

```
PS E:\Github rep\DevelopmentURFU2025\LR2\main> python -m alembic revision --autogenerate -m "init tables"
INFO [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO [alembic.runtime.migration] Will assume transactional DDL.
INFO [alembic.autogenerate.compare] Detected added table 'users'
INFO [alembic.autogenerate.compare] Detected added table 'addresses'
Generating E:\Github rep\DevelopmentURFU2025\LR2\main\migrations\versions\787eba9cba7f_init_tables.py ... do
PS E:\Github rep\DevelopmentURFU2025\LR2\main> python -m alembic upgrade head
INFO [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO [alembic.runtime.migration] Will assume transactional DDL.
INFO [alembic.runtime.migration] Running upgrade -> 787eba9cba7f, init tables
PS E:\Github rep\DevelopmentURFU2025\LR2\main>
```

Рисунок 4 – Работа с миграциями

Проверили корректность работы (Рисунок 5).

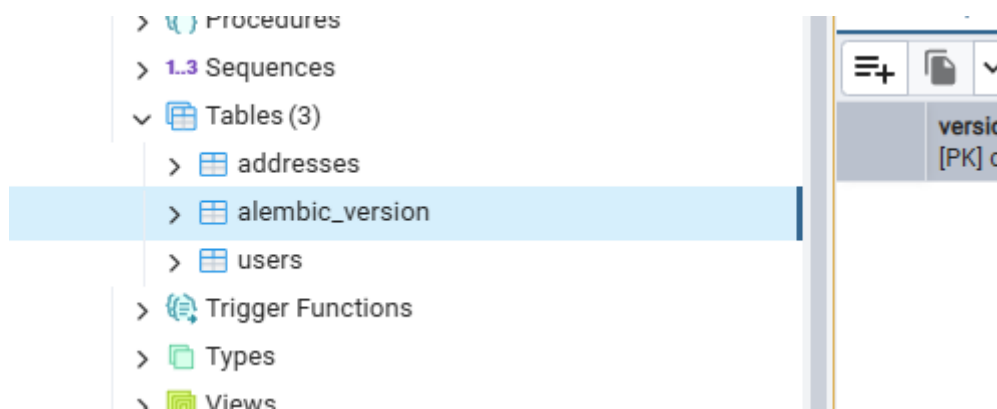


Рисунок 5 – Подключение к БД внутри контейнера

3. Наполнение БД данными

Создадим скрипт для наполнения БД «seed_data.py» и запустим его командой «python seed_data.py», в результате чего БД наполнилась 5 пользователями и их адресами (Рисунок 6).

```
PS E:\bit\ufp\wp\Development\URFU2025\LR2\main> python seed_data.py
2025-12-16 21:23:27,364 INFO sqlalchemy.engine.Engine select pg_catalog.version()
2025-12-16 21:23:27,365 INFO sqlalchemy.engine.Engine [raw sql] {}
2025-12-16 21:23:27,367 INFO sqlalchemy.engine.Engine select current_schema()
2025-12-16 21:23:27,367 INFO sqlalchemy.engine.Engine [raw sql] {}
2025-12-16 21:23:27,368 INFO sqlalchemy.engine.Engine show standard_conforming_strings
2025-12-16 21:23:27,369 INFO sqlalchemy.engine.Engine [raw sql] {}
2025-12-16 21:23:27,350 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-12-16 21:23:27,352 INFO sqlalchemy.engine.Engine INSERT INTO users (id, username, email, description, created_at, updated_at) VALUES (%(id_0)s::UUID, %(username_0)s, %(email_0)s, %(description_0)s, %(created_at_0)s,
updated_at_0)s, (%(id_1)s::UUID, %(username_1)s, %(email_1)s, %(description_1)s, %(created_at_1)s, %(updated_at_1)s)
2025-12-16 21:23:27,352 INFO sqlalchemy.engine.Engine [generated in 0.00014s (insertmanyvalues) 1/1 (unordered)] {'updated_at_0': None, 'description_0': None, 'id_0': UUID('127d8bc7-70ec-4257-8762-5e9a5a90a0d5'), 'email_0':
'ilya@example.com', 'username_0': 'ilya', 'created_at_0': datetime.datetime(2025, 12, 16, 21, 23, 27, 352447), 'updated_at_1': None, 'description_1': None, 'id_1': UUID('71039e37-fb0b-45c0-a9b5-9518edf59922'), 'email_1':
'misha@example.com', 'username_1': 'misha', 'created_at_1': datetime.datetime(2025, 12, 16, 21, 23, 27, 352447), 'updated_at_2': None, 'description_2': None, 'id_2': UUID('57e8e819-64e3-44b5-a29a-bc5c84363394'), 'email_2':
'liding@example.com', 'username_2': 'lidi', 'created_at_2': datetime.datetime(2025, 12, 16, 21, 23, 27, 352447), 'updated_at_3': None, 'description_3': None, 'id_3': UUID('cc098814-ea04-4757-b100-567d3ca05e53'), 'email_3':
'artem@example.com', 'username_3': 'artem', 'created_at_3': datetime.datetime(2025, 12, 16, 21, 23, 27, 352447), 'updated_at_4': None, 'description_4': None, 'id_4': UUID('70efffc7-4a08-4308-a9a3-203082237051'), 'email_4':
'slava@example.com', 'username_4': 'slava', 'created_at_4': datetime.datetime(2025, 12, 16, 21, 23, 27, 352447)}
2025-12-16 21:23:27,359 INFO sqlalchemy.engine.Engine COMMIT
2025-12-16 21:23:27,362 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-12-16 21:23:27,364 INFO sqlalchemy.engine.Engine SELECT users.id AS users_id, users.username AS users_username, users.email AS users_email, users.description AS users_description, users.created_at AS users_created_at, us
.updated_at AS users_updated_at
FROM users
WHERE users.id = %(pk_1)s::UUID
2025-12-16 21:23:27,365 INFO sqlalchemy.engine.Engine [generated in 0.00012s] {'pk_1': UUID('127d8bc7-70ec-4257-8762-5e9a5a90a0d5')}
2025-12-16 21:23:27,368 INFO sqlalchemy.engine.Engine SELECT users.id AS users_id, users.username AS users_username, users.email AS users_email, users.description AS users_description, users.created_at AS users_created_at, us
.updated_at AS users_updated_at
```

Рисунок 6 – Выполненная команда

4. Запрос связанных данных

Создали файл «query_data.py» и запустим его командой «python query_data.py» (Рисунок 7).

```
PS E:\Github rep\DevelopmentURFU2025\LR2\main> python query_data.py
Пользователь: ilya (ilyae@example.com)
Адрес: 10 street, Revda, RU
-----
Пользователь: misha (misha@example.com)
Адрес: 20 street, Los Angeles, USA
-----
Пользователь: lildi (lildin@example.com)
Адрес: 30 street, Chicago, USA
-----
Пользователь: artem (artem@example.com)
Адрес: 40 street, Madrid, Spain
-----
Пользователь: slava (slava@example.com)
Адрес: 50 street, Beijing, China
-----
PS E:\Github rep\DevelopmentURFU2025\LR2\main>
```

Рисунок 7 – Вывод данных о каждом пользователе

5. Модификация БД

В файле «models.py» добавили поле «description» в класс User (Рисунок 8). А также добавили новые таблицы «Product» и «Order» и чуть не забыли про связи! (Рисунок 9).

```
class User(Base):  
    __tablename__ = 'users'  
  
    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)  
    username = Column(String, nullable=False, unique=True)  
    email = Column(String, nullable=False, unique=True)  
    description = Column(String, nullable=True)  
    created_at = Column(DateTime, default=datetime.now)  
    updated_at = Column(DateTime, onupdate=datetime.now)  
  
    addresses = relationship("Address", back_populates="user")
```

Рисунок 8 – Добавление поля «description»


```

class Product(Base): new *
    __tablename__ = 'products'

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    name = Column(String, nullable=False)
    price = Column(Numeric(precision=10, scale=2), nullable=False)
    description = Column(String)
    created_at = Column(DateTime, default=datetime.now)

class Order(Base): new *
    __tablename__ = 'orders'

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    user_id = Column(UUID(as_uuid=True), ForeignKey('users.id'), nullable=False)
    address_id = Column(UUID(as_uuid=True), ForeignKey('addresses.id'), nullable=False)
    product_id = Column(UUID(as_uuid=True), ForeignKey('products.id'), nullable=False)
    quantity = Column(Integer, default=1)
    status = Column(String, default="pending")
    created_at = Column(DateTime, default=datetime.now)

    user = relationship("User")
    address = relationship("Address")
    product = relationship("Product")

```

Рисунок 9 – Добавление таблиц «Product» и «Order»

Далее, чтобы сгенерировать миграцию, прописали команду «python -m alembic revision --autogenerate -m "add description to user, add products and orders"» и применили миграцию командой «python -m alembic upgrade head»

Наполним БД новыми данными, создав файл «seed_data_2.py» и запустив его командой «python seed_data_2.py». Проверим изменения (Рисунок 10).

Query

Query History

↗

Scratch Pad ×

1

SELECT * FROM public.products

2

ORDER BY id ASC

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

📦

⬇️

📈

Showing rows: 1 to 5

Page No: 1 of 1

⏪

⏩

	id [PK] uuid	name character varying	price numeric (10,2)	description character varying	created_at timestamp without time zone
1	4c2b496f-746d-4e86-a2f7-fa7e9ea1c59e	Клавиатура	599.99	Клава с подсветкой	2025-12-16 21:34:12.781746
2	83eece19-6aa4-4c02-adfd-c94e3ab051...	Наушники	3499.00	Шумоподавление	2025-12-16 21:34:12.781746
3	9b38efd7-76c8-4bea-82e2-527649887b...	Книга	889.50	Крутая книга	2025-12-16 21:34:12.781746
4	ca17966b-6a96-40dd-8344-20b2b5094...	Футболка с берсерком	1122.00	Хлопок, размер XL	2025-12-16 21:34:12.781746
5	f99bebf9-cca4-46ab-8cb6-2b4232410213	Мышка	12999.00	Крутая	2025-12-16 21:34:12.781746

Рисунок 10 – Новые данные

Вопросы:

1. Какие есть подходы маппинга в SQLAlchemy? Когда следует использовать каждый подход?

- Декларативный - самый простой: требует написания класса, после этого всё сразу понятно.
- Классический - сначала нужно описать таблицу, потом класс, потом связать их.
- Автомаппинг - когда имеется база, а SQLAlchemy сам придумывает классы

В данной работе мы использовали первый, потому что он самый лёгкий.

2. Как Alembic отслеживает текущую версию базы данных?

Alembic создаёт в базе специальную служебную таблицу под названием «alembic_version». В ней хранится одна строчка - аналог «ярлыка», который хранит ID последней применённой миграции. При запуске «alembic upgrade head», Alembic смотрит, какая версия сейчас в таблице, какие миграции есть в папке versions и применяет только те, которых ещё нет.

3. Какие типы связей между таблицами вы реализовали в данной работе?

Мы сделали связь «один-ко-многим»: один пользователь может иметь несколько адресов, но каждый адрес принадлежит только одному пользователю. В таблице «addresses» есть поле «user_id», которое ссылается на «id» в таблице «users». В коде мы использовали ForeignKey и relationship, чтобы SQLAlchemy знал, как эти таблицы связаны. Связи «многие ко многим» (например, один пользователь - много продуктов, и один продукт - у многих

пользователей) у нас не было, но можно было бы добавить, если бы за один заказ можно было бы купить несколько товаров.

4. Что такое миграция базы данных и почему она важна?

Миграция - файл с инструкциями, как изменить структуру базы данных: добавить таблицу, колонку, изменить тип и т.д. Она важна, потому что без неё пришлось бы вручную редактировать базу на каждом компьютере, а с миграциями же все изменения сохраняются в коде. В случае ошибки можно откатиться назад.

5. Как обрабатываются отношения многие-ко-многим в SQLAlchemy

Связь «многие ко многим» (например, «пользователь может покупать много товаров, и товар может быть у многих пользователей») нельзя сделать напрямую в реляционной базе. Нужна дополнительная таблица-посредник.

Вывод:

В ходе выполнения лабораторной работы были освоены основы работы с библиотеками SQLAlchemy и Alembic для управления реляционной базой данных в Python.

Были созданы ORM-модели пользователей, адресов, продуктов и заказов, настроены связи между ними и запущены PostgreSQL в Docker без установки СУБД на компьютер.

С помощью Alembic я были сгенерированы и применены миграции, что позволило безопасно изменять структуру базы данных без ручного вмешательства.

При наполнении БД тестовыми данными и выполнении запросов к связанным таблицам убедились, что всё работает корректно.

Работа показала, насколько удобно и надёжно можно управлять базой данных через код: легко воспроизводятся и не зависят от конкретного окружения. Полученные навыки могут быть использованы в будущих проектах для построения гибких и поддерживаемых приложений.

Ссылка на репозиторий:

<https://github.com/orfimorf/DevelopmentURFU2025>